

dataQ 표준 진단 실행 로직: READY 상태와 중복 클릭

시점: 2026-04-초 (진단 기능이 한 번 안정화된 뒤, 운영 중 반복 재현된 문제를 파고든 기록)

문제가 드러난 방식

표준화 진단은 q-center에서 "시작" 버튼을 누르면 q-executor로 비동기 요청이 가고, executor가 스레드를 띄워 컬럼을 하나씩 훑는다. 프론트는 3초마다 Job 상태를 폴링해서 화면에 반영한다. 여기까지는 설계대로 움직였다.

문제는 두 가지였다.

1. 진단을 시작했는데 화면의 상태가 **READY** 에 머물러 있는 경우가 가끔 나왔다. 처리 건수도 0으로 고정.
2. 같은 수집건에 대해 Job이 두 개 이상 생기는 경우가 있었다. 진단 결과가 중복으로 쌓였다.

둘 다 "가끔" 나오는 버그였고, 재현이 어려웠다. 한동안 "왜 일어나는지"를 추정만 하고 있다가, 실행 흐름을 그림으로 다시 그려보면서 두 버그가 사실은 같은 뿌리에서 나왔다는 걸 확인했다.

현재의 실행 흐름

```
[프론트 DSDataDiag.vue]
  "진단 시작" 클릭
    ↓
  POST /api/diag/startDiag
    ↓
[q-center DiagController]
  ① TB_DIAG_JOB INSERT (status='READY')
  ② q-executor에 WebClient 비동기 요청
    ↓
[q-executor DiagController]
  Thread.start() → DiagService.run()
    ↓
[DiagService - 별도 스레드]
  status → 'RUNNING' (START_DT 기록)
  컬럼별 루프 (50건마다 flush)
  status → 'DONE' (END_DT 기록)
    ↑
[프론트 3초 폴링]
  GET /api/diag/getDiagJobById
  DONE/ERROR 수신 시 폴링 종료
```

READY는 상태 머신상 "Job 레코드는 만들어졌지만 executor 스레드가 아직 status를 RUNNING으로 바꾸지 않은 구간"을 가리킨다. 정상 흐름에서는 1~3초 머물다가 RUNNING으로 넘어간다. 프론트가 그 사이에 폴링하면 READY가 잠깐 보이는 것이 정상이다.

그런데 "영구 READY"가 생기는 경로가 여럿 있었다.

READY에 갇히는 네 가지 경로

경로	무엇이 일어나는가
q-executor 연결 실패	q-center가 executor에 HTTP 요청을 보냈으나 타임아웃·네트워크 오류. Job은 ①에서 이미 INSERT됐고, executor는 받지도 못함.
executor 과부하	Thread.start()는 성공했지만 CPU/메모리 부족으로 스레드가 즉시 실행되지 못하고 큐에서 대기.
executor 미기동	q-executor 프로세스가 내려가 있는 상태에서 시작 버튼을 누르면 ②가 실패.
DB 접속 실패	executor 스레드가 났지만 <code>updateStatus("RUNNING")</code> 쿼리 자체가 예외. 스레드는 종료되지만 status는 READY인 채로 남음.

네 경로의 공통점은 하나다. ①은 성공하고 ② 이후가 실패하는데, 실패 시 Job status를 되돌리거나 ERROR로 바꾸는 로직이 없다. 현재 코드는 이렇다.

```
// q-center DiagController.startDiag()
sqlSessionTemplate.insert("diag.insertDiagJob", jobVo); // ① READY로 INSERT
return webClientHandler.postData(params)                // ② executor 호출
    .map(res -> { ... });
```

②에서 `onErrorResume` 이 없다. executor 호출이 Mono의 에러로 떨어지면 프론트로 500이 나가고 끝이다. DB에 남은 READY Job은 손대지 않는다. 좀비 Job.

중복 클릭은 왜 뚫렸는가

프론트의 방어 로직은 이랬다.

```
<v-btn :disabled="!selectedClctId || isRunning" @click="startDiag">
```

```
isRunning() {
  return this.currentJob && this.currentJob.status === 'RUNNING';
}
```

`isRunning` 은 폴링으로 받아온 `currentJob.status` 가 `RUNNING` 일 때만 true다. 타임라인으로 풀면 이렇게 된다.

T=0ms	클릭 → startDiag() 호출, axios 요청 시작
T=50ms	요청 진행 중. 아직 응답 없음. currentJob 미갱신. → isRunning = false, 버튼 여전히 활성화
T=100ms	사용자가 또 클릭 → startDiag() 또 호출 ← 여기서 중복 발생
T=200ms	첫 요청 응답 → Job1 INSERT (READY)
T=300ms	두 번째 요청 응답 → Job2 INSERT (READY)
T=1s	Job1 → RUNNING, Job2 → RUNNING (병렬)

두 가지가 겹친 것이다.

1. isRunning 이 RUNNING만 체크하고 READY는 통과시킨다.
2. API 응답 전에는 currentJob 자체가 갱신 안 되어 비활성화 조건이 아무 것도 걸리지 않는다.

첫 클릭과 두 번째 클릭 사이 100~300ms 구간이 완전히 무방비였다. 이 구간을 "설마 그 짧은 사이에"라고 지나친 것이 잘못이었다. 실제로 조바심 많은 사용자는 그 구간을 정확히 찾는다.

READY 상태의 의미를 다시 정의

여기서 깨달은 건 코드 문제라기보다 상태의 정의가 흐릿했다는 것이다. READY 는 처음에 "데이터는 로드됐지만 처리 시작 전"이라는 모호한 뜻으로 두었다. 그런데 위 분석을 하고 나면 READY가 두 가지 전혀 다른 상태를 동시에 의미하고 있었다는 게 드러난다.

- **정상 READY:** executor가 곧 RUNNING으로 바꿀 1~3초짜리 과도 상태.
- **좀비 READY:** executor가 도달하지 못했거나 실패해서 영구히 머무는 상태.

화면에서는 둘을 구분할 방법이 없다. 똑같이 "대기 중"으로 보인다. 사용자 입장에서는 "왜 안 돌지?" 하고 다시 누르게 된다. 다시 누르면 중복 Job이 생긴다. 두 버그가 같은 뿌리라는 건 이 지점이다. 좀비 READY가 중복 클릭을 유발하고, 중복 클릭이 또 다른 좀비 READY를 만든다.

개선 방향과 우선순위

1. **프론트 starting 플래그 (필수·간단).** API 호출 시작~응답 수신 사이에 별도 플래그를 두고 버튼을 비활성화한다. 코드 5줄이고, 100~300ms 구간 무방비를 막는다.

```
js data: { starting: false }, startDiag() { if (this.starting) return; this.starting = true;
axios.post(...).finally(() => { this.starting = false; }); } <v-btn
:disabled="!selectedClctId || isRunning || starting">
```

2. **executor 호출 실패 시 Job → ERROR (필수·간단).** ②의 Mono에 onErrorResume 을 달아서 실패 시 Job status를 ERROR로 바꾸고 end_dt를 찍는다. 좀비 READY가 남지 않는다.

```
java .onErrorResume(e -> { jobVo.setStatus("ERROR");
sqlSessionTemplate.update("diag.updateDiagJobStatus", jobVo); return
Mono.just(Results.error500()); });
```

3. **백엔드에서도 중복 Job 차단 (권장).** 같은 clctId에 READY/RUNNING 상태 Job이 있으면 409로 거부한다. 프론트 플래그는 뚫릴 수 있으니(탭을 두 개 연 경우 등) 서버에도 게이트를 둔다.

시급도 순이다. 1, 2는 작은 변경이고 3은 쿼리 하나 추가.

돌아보고 남긴 것

- 상태 머신을 설계할 때 **"정상 전이 중 찰나"**와 **"비정상 정지"**를 같은 상태명으로 묶지 않는다. READY를 두 상태로 쓴 게 모든 혼란의 출발이었다. 필요하다면 상태를 쪼개거나, 최소한 "얼마나 머물러 있었는가"를 함께 표시한다.
- 비동기 체인의 에러 경로는 **INSERT** 경로만큼 **공들여 그려야 한다**. 성공 경로만 그리고 끝내면 실패시의 DB 상태는 대부분 "정의되지 않음"이 된다. 여기서 좀비가 나온다.
- **"이 짧은 구간에 설마"**는 **정확히 재현된다**. 사용자가 0.1초를 찌르는 일은 자주 있다. 프론트 방어는 "눈에 보이는 상태가 바뀌기 전"에도 걸려야 한다.